



Section 1 / 24

[Go to Questions](#) ↓

Assembly Language

importance of low-level with the need to translate from high-level to low-level

Most of our interaction with our personal computers and smartphones is done through the operating system and other applications. These applications are usually developed using high-level languages, like C++, Java, Python, and many others. We also know that each of these devices has a core processor that runs all of the necessary processes to execute systems and applications, along with Random Access Memory (RAM), Video Memory, and other similar components.

However, **these physical components cannot interpret or understand high-level languages**, as they can essentially only process **1** 's and **0** 's. This is where Assembly language comes in, as a low-level language that can write direct instructions the processors can understand. Since the processor can only process binary data "i.e. **1** 's and **0** 's", it would be challenging for humans to interact with processors without referring to manuals to know which hex code runs which instruction.

This is why low-level assembly languages were built. By using Assembly, developers can **write human-readable machine instructions**, which are then assembled into their machine code equivalent, so that the processor can directly run them. This is why some refer to Assembly language as **symbolic machine code**. For example, the Assembly code `' add rax, 1 '` is much more intuitive and easier to remember than its equivalent machine shellcode `' 4883C001 '`, and

easier to remember than the equivalent binary machine code ' `01001000 10000011 11000000 00000001` '. As we can see, without Assembly language, it is very challenging to write machine instructions or directly interact with the processor.

Machine code is often represented as `Shellcode` , a hex representation of machine code bytes. Shellcode can be translated back to its Assembly counterpart and can also be loaded directly into memory as binary instructions to be executed.

High-level vs. Low-level

As there are different processor designs, each processor understands a different set of machine instructions and a different Assembly language. In the past, applications had to be written in assembly for each processor, so it was not easy to develop an application for multiple processors. In the early 1970's, high-level languages (like `C`) were developed to make it possible to write a single easy to understand code that can work on any processor without rewriting it for each processor. To be more specific, this was made possible by creating compilers for each language.

When high-level code is compiled, it is translated into assembly instructions for the processor it is being compiled for, which is then assembled into machine code to run on the processor. This is why compilers are built for various languages and various processors to convert the high-level code into assembly code and then machine code that matches the running processor.

Later on, interpreted languages were developed, like `Python` , `PHP` , `Bash` , `JavaScript` , and others, which are usually not compiled but are interpreted during run time. These types of languages utilize pre-built libraries to run their instructions. These libraries are typically written and compiled in other high-level languages like `C` or `C++` . So when we issue a command in an interpreted language, it would use the compiled library to run that command, which uses its

assembly code/machine code to perform all the instructions necessary to run this command on the processor.

Compilation Stages

Interpreted Language 'Python, JS, Bash'	High Level Language 'C, C++'	Low Level Language 'Assembly'	Hex Machine Code	Binary Machine Code
print("Hello World!")	write(1,"Hello World!",12); _exit(0);	mov rax, 1 mov rdi, 1 mov rsi, message mov rdx, 12 syscall mov rax, 60 mov rdi, 0 syscall	48 c7 c0 01 48 c7 c7 01 48 8b 34 25 48 c7 c2 0c 0f 05 48 c7 c0 3c 48 c7 c7 00 0f 05	01001000 11000111 11000000 00000001 01001000 11000111 11000111 00000001 01001000 10001011 00110100 00100101 01001000 11000111 11000010 00001100 00001111 00000101 01001000 11000111 11000000 00111100 01001000 11000111 11000111 00000000 00001111 00000101

Let's take a basic 'Hello World!' program that prints these words on the screen and show how it changes from high-level to machine code. In an interpreted language, like Python, it would be the following basic line:

Python

```
print("Hello World!")
```

If we run this Python line, it would be essentially executing the following C code:

C

```
#include <unistd.h>

int main()
{
    write(1, "Hello World!", 12);
    _exit(0);
}
```

Note: the actual C source code is much longer, but the above is the essence of how the string 'Hello World!' is printed. If you are ever interested in knowing more, you can check out the source code of the Python3 print function at this [link](#) and this [link](#)

The above C code uses the Linux `write` syscall, built-in for processes to write to the screen. The same syscall called in Assembly looks like the following:

```
nasm
mov rax, 1
mov rdi, 1
mov rsi, message
mov rdx, 12
syscall

mov rax, 60
mov rdi, 0
syscall
```

As we can see, when the `write` syscall is called in C or Assembly, both are using `1`, the text, and `12` as the arguments. This will be covered more in-depth later in the module. From this point, Assembly code, shellcode, and binary machine code are mostly identical but written in different formats. The previous Assembly code can be assembled into the following hex machine code (i.e., shellcode):

```
shellcode
48 c7 c0 01
48 c7 c7 01
48 8b 34 25
```

```
48 c7 c2 0c
0f 05
```

```
48 c7 c0 3c
48 c7 c7 00
0f 05
```

Finally, for the processor to execute the instructions linked to this machine, it would have to be translated into binary, which would look like the following:

```
01001000 11000111 11000000 00000001
01001000 11000111 11000111 00000001
01001000 10001011 00110100 00100101
01001000 11000111 11000010 00001100
00001111 00000101

01001000 11000111 11000000 00111100
01001000 11000111 11000111 00000000
00001111 00000101
```

binary

A CPU uses different electrical charges for a **1** and a **0**, and hence can calculate these instructions from the binary data once it receives them.

Note: With multi-platform languages, like **Java**, the code is **compiled into a Java Bytecode**, which is the same for all processors/systems, and is then **compiled to machine code** by the **local Java Runtime environment**. *This is what makes Java relatively slower than other*

languages like C++ that compile directly into machine code. Languages like C++ are more suitable for processor intensive applications like games.

We now see how computer languages progressed from assembly language unique for each processor to high-level languages that can work on any device without even needing to be compiled.

Value for Pentesters

Understanding assembly language instructions is critical for binary exploitation, which is an essential part of penetration testing. When it comes to exploiting compiled programs, the only way to attack them would be through their binaries. To disassemble, debug, and follow binary instructions in memory and find potential vulnerabilities, we must have a basic understanding of Assembly language and how it flows through the CPU components.

This is why once we start learning binary exploitation techniques, like buffer overflows, ROP chains, heap exploitation, and others, we will be dealing a lot with assembly instructions and following them in memory. Furthermore, to exploit these vulnerabilities, we will have to build custom exploits that use assembly instructions to manipulate the code while in memory and inject assembly shellcode to be executed.

Learning Intel x86 Assembly Language is crucial for writing exploits for binaries on modern machines. In addition to Intel x86, ARM is becoming more common, as most modern smartphones and some modern laptops like the M1 MacBook Pro feature ARM processors. Exploiting binaries in these systems requires ARM Assembly knowledge. This module will not cover ARM Assembly Language. That being said, Assembly Language basics will undoubtedly be helpful to anyone willing to learn ARM Assembly since the two languages have a lot of similarities.



Enable step-by-step solutions ⓘ

PRO

Optional Exercise 1

XP +40 ^

In the above 'Hello World' example, which Assembly instruction will '00001111 00000101' execute?

Write your answer

Submit

💡 Reveal Answer

Section 1 / 24

XP +20

Next

Mark Complete & Next